基于深度学习的香蕉新鲜度等级识别系统

《深度学习与计算机视觉》期末考评自主创作大作业

专　业　：软件工程

方　向　：人工智能

课程名称：深度学习与计算机视觉

班　级　：22130401

学　号　：2213040106

姓　名　：梁蓝

辅导教师：井超

摘要

本文提出了一种基于 YOLOv10 的香蕉新鲜度等级识别系统，该系统利用计算机视觉技术实现对香蕉成熟度的自动识别与分类。创新性地精准区分出六种成熟度类别：新鲜成熟、新鲜未成熟、过熟、成熟、腐烂和未成熟。借助 YOLOv10 模型强大的实时检测能力，在包含 18,074 张图像的数据集上完成训练、验证及测试。实验结果表明，该系统在香蕉成熟度检测任务中性能出色，具有较高的准确率和强大的鲁棒性，为香蕉品质评估与相关产业应用提供了有力的技术支撑，可帮助解决香蕉拍卖和营销中的挑战，提高销售效率。

关键词：YOLOv10；计算机视觉；自动识别与分类

Abstract

This paper proposes a banana freshness class recognition system based on YOLOv10, which uses computer vision technology to achieve automatic recognition and classification of banana ripeness. Six ripeness categories are innovatively and accurately distinguished: fresh ripe, fresh unripe, overripe, ripe, rotten and unripe. With the powerful real-time detection capability of the YOLOv10 model, the system was trained, validated and tested on a dataset of 18,074 images. The experimental results show that the system performs well in the banana ripeness detection task with high accuracy and strong robustness, which provides a strong technical support for banana quality assessment and related industrial applications, and can help to solve the challenges in banana auctioning and marketing, and improve the efficiency of sales.

Keywords: YOLOv10; Computer vision technology; Automatic recognition and classification

目录

1 引言 ····· 1

1.1 背景 ····· 1

1.2 目的 ····· 1

2 相关技术介绍 ····· 2

2.1 YOLOv10 实现端到端目标检测技术 ····· 2

2.2 PyQt5 跨平台 Python GUI 开发框架 ····· 3

3 数据集介绍 ····· 5

3.1 数据集概述 ····· 5

3.2 类别分布与特点 ····· 5

3.3 数据预处理 ····· 6

3.4 数据集特点与适用性 ····· 6

3.5 数据集配置文件 ····· 7

4 环境配置 ····· 8

4.1 虚拟环境 ····· 8

4.2 安装所需库 ····· 8

5 系统架构设计 ····· 10

6 详细设计 ····· 11

6.1 模型训练 ····· 11

6.1.1 模型结构和调整 ····· 11

6.1.2 训练代码 ····· 11

6.1.3 训练结果 ····· 12

6.2 核心部分 ····· 15

6.2.1 登录模块（LoginWindow） ····· 15

6.2.2 主功能模块(MainWindow) ····· 20

7 结果展示 ····· 27

7.1 登陆窗口 ····· 27

7.2 主功能窗口 ····· 28

8 项目成果总结 ····· 33

1 引言

1.1 背景

香蕉作为一种广泛受欢迎的水果，其新鲜度直接关系到口感品质与营养价值，进而影响市场价值和消费者的购买意愿。在香蕉的种植、采摘、运输、销售等各个环节，对香蕉新鲜度的准确判断都具有重要意义。传统的香蕉新鲜度检测方法往往依赖人工经验，存在主观性强、效率低下、难以量化等问题，难以满足现代香蕉产业大规模、快速流通的需求。随着计算机视觉技术和深度学习的飞速发展，利用先进的算法模型实现对香蕉新鲜度的自动识别成为了可能。YOLO 系列目标检测算法以其高效、精准的特点在众多计算机视觉任务中脱颖而出，而 YOLOv10 作为该系列的最新成果，具备更强大的性能和适应能力，为解决香蕉新鲜度自动识别这一实际问题提供了有力的技术工具。

1.2 目的

本系统旨在详细介绍基于 YOLOv10 的香蕉新鲜度等级识别系统。该系统的构建目的在于研发一套能够高效、准确地对香蕉新鲜度进行自动分类的智能解决方案，以克服传统人工检测方法的局限性，满足香蕉产业链各环节对新鲜度快速、精准评估的需求。通过在大规模图像数据集上的训练与优化，使系统能够精准区分新鲜成熟、新鲜未成熟、过熟、成熟、腐烂、未成熟这六种香蕉成熟度类别，并实现实时检测效果，为香蕉品质的自动化评估提供可靠依据，从而推动香蕉产业的智能化发展，助力提升产业整体效益和市场竞争力。

2 相关技术介绍

2.1 YOLOv10 实现端到端目标检测技术

(1) 核心创新与架构优化

YOLOv10 由清华大学研究团队基于 Ultralytics 框架开发，是 YOLO 系列的最新版本，专注于解决传统 YOLO 模型在后处理冗余和架构效率上的不足。其核心创新包括：

无 NMS（非极大值抑制）训练：通过“一致双重分配策略”，在训练时同时使用一对多和一对一预测头。一对多头提供丰富的监督信号，一对一头则在推理时直接输出最优预测，无需 NMS 无需 NMS 后处理，显著降低延迟并实现端到端部署。

整体效率-准确性驱动设计：

- 主干网络：采用增强版 CSPNet（跨阶段部分网络），优化梯度流动并减少计算冗余。
- 颈部结构：集成路径聚合网络（PAN），提升多尺度特征融合能力。
- 轻量化组件：引入轻量级分类头、空间-通道解耦下采样，以及基于秩引导的模块设计（CIB），减少计算开销。
- 增强模型能力：大核卷积扩大感受野，部分自注意力模块（PSA）结合全局与局部信息，提升检测精度。

(2) 模型变体与性能优势

YOLOv10 提供六种不同规模的模型，覆盖广泛的应用场景：

YOLOv10-N/S/M/B/L/X：从纳米级（N）到超大型（X），兼顾资源受限环境与高精度需求。

性能表现（以 COCO 数据集为例）：

- 速度与精度：YOLOv10-S 在相似平均精度（AP）下比 RT-DETR-R18 快 1.8 倍；YOLOv10-B 相比 YOLOv9-C 减少 46%延迟和 25%参数。
- 参数效率：YOLOv10-L/X 的参数量比 YOLOv8-L/X 少 1.8/2.3 倍，AP

值提升 0.3/0.5。

(3) 应用场景与部署优势

实时检测：适用于自动驾驶、视频监控等低延迟场景，支持端到端部署，无需复杂后处理。

灵活适配：通过模型缩放策略，用户可根据硬件资源选择合适版本，如 YOLOv10-N 适用于嵌入式设备，YOLOv10-X 适用于服务器端高精度检测。

开发便捷性：支持 PyTorch 和 ONNX 格式导出，并提供与 Ultralytics 框架兼容的训练接口，便于快速集成与迁移学习。

(4) 技术对比与实验验证

在标准基准测试中，YOLOv10 在不同规模下均超越 YOLO 系列前代及 RT-DETR 等竞品。例如，YOLOv10-X 在保持与 RT-DETR-R101 相似 AP 的同时，推理速度提升 1.3 倍，参数减少近 2 倍。

2.2 PyQt5 跨平台 Python GUI 开发框架

(1) 概述

PyQt5 是 Python 语言与 Qt 应用框架的结合体，由 Riverbank Computing 开发，基于 Qt5.x 版本构建。它提供了完整的 Qt 功能库和工具链，支持快速开发跨平台桌面应用程序，广泛应用于数据可视化、工业控制、嵌入式界面及企业级软件开发。

(2) 核心技术与功能亮点

跨平台支持：支持 Windows、macOS、Linux、Android 等操作系统，一次开发多端部署，适配性强。

丰富的组件库：包含 600+预置控件（如按钮、表格、图表、3D 视图等），支持自定义样式（QSS）与主题切换。

信号与槽机制：基于事件驱动的通信模型，通过信号（Signal）与槽（Slot）实现组件间解耦交互，简化复杂逻辑设计。

Qt Designer 可视化开发：提供图形化界面设计工具，支持拖拽式布局，

生成的.ui 文件可直接转换为 Python 代码，提升开发效率。

多线程与异步处理：集成 QThread 和 QThreadPool，优化耗时任务处理，避免界面卡顿。

多媒体与硬件交互：支持 OpenGL 渲染、摄像头调用、串口通信、网络协议栈等高级功能。

(3) 版本与生态对比

PyQt5 vs. PySide2：PyQt5 采用 GPL/商业双协议，功能更新更快；PySide2（Qt 官方 Python 绑定）使用 LGPL 协议，适合开源项目。

与其他 GUI 框架对比：

- Tkinter：PyQt5 功能更强大，界面更现代化；
- wxPython：PyQt5 性能更优，文档更完善；
- Kivy：PyQt5 更适合传统桌面应用，而非移动端优先场景。

(4) 典型应用场景

数据可视化工具：结合 Matplotlib 或 PyQtGraph 库，构建交互式图表分析界面。

工业控制面板：通过串口/网络协议与硬件设备通信，实现实时监控与控制。

跨平台桌面软件：如音视频播放器、办公自动化工具、科学计算平台等。

嵌入式界面开发：适配树莓派等嵌入式设备，驱动触摸屏与传感器。

(5) 开发优势

高效开发：Python 语法简洁，配合 Qt Designer 快速原型设计。

可扩展性：无缝集成 NumPy、OpenCV 等 Python 科学计算库，扩展功能模块。

维护便捷：代码结构清晰，支持 MVVM 设计模式，便于团队协作与后期升级。

社区支持：拥有活跃的开源社区和丰富的第三方插件（如 QDarkTheme、PyQtGraph）。

3 数据集介绍

3.1 数据集概述

本研究所采用的数据集由 18,074 张香蕉图像构成，全面覆盖了新鲜成熟、新鲜未成熟、过熟、成熟、腐烂、未成熟这六种不同的成熟度类别。

按照机器学习的通用实践，数据集被精心划分为三个部分：训练集、验证集和测试集，具体分布为训练集 15,792 张图像，用于模型的参数学习与优化；验证集 1,525 张图像，用于在训练过程中对模型进行中期评估，辅助调整训练策略；以及测试集 757 张图像，用于在模型训练完成后进行最终的性能评估，确保模型的泛化能力（数据接近于 10:1:1）。

3.2 类别分布与特点

数据集中的六种成熟度类别各具特点：

- **新鲜成熟（freshripe）**：香蕉达到理想成熟度，外观呈现鲜亮的黄色，表皮光滑且富有弹性，无明显瑕疵，这是香蕉口感与营养的最佳时期，也是消费者普遍偏好的状态。
- **新鲜未成熟（freshunripe）**：香蕉尚未完成成熟过程，外观以绿色为主，果肉质地坚硬，口感较生涩，但储存时间相对较长，便于运输与后续销售中的自然成熟调控。
- **过熟（overripe）**：香蕉过度成熟，外观颜色加深至深黄色，表面出现大量褐色斑点，果肉质地变软，甜度达到峰值，但已经开始出现腐坏的前兆，若不及时处理则会迅速变质。
- **成熟（ripe）**：这是香蕉的标准成熟状态，外观黄色均匀，果肉质地适中，口感香甜且易于消化，是市场销售的主要目标状态之一，兼具口感与相对稳定的储存期。
- **腐烂（rotten）**：香蕉已处于变质阶段，外观呈现深褐色或黑色，果肉软

烂，表面可能有明显的霉变痕迹或液体渗出，散发异味，完全失去食用价值，是需要在产业链中严格剔除的部分。

- **未成熟（unripe）**：与新鲜未成熟类似，也呈现绿色，质地硬，但相比新鲜未成熟，其在某些生理特性或储存潜力上可能略有差异，需通过数据驱动的分类来明确区分。

3.3 数据预处理

在将数据集用于训练 YOLOv10 模型之前，为提升模型性能与泛化能力，进行了以下预处理步骤：

(1) **图像增强**：采用随机旋转、缩放、翻转等操作对图像进行变换，模拟不同拍摄角度、距离和方向的香蕉图像，以此增加数据的多样性。这种处理方式使模型能够接触到更广泛的图像特征组合，从而在面对实际场景中各种复杂情况时，依然能够准确识别香蕉的成熟度，有效提高模型对不同光照条件、背景环境的适应性，增强其鲁棒性。

(2) **标注文件制作**：每张图像都配备详细的标注文件，标注文件中精确记录了香蕉的边界框信息以及对应的类别标签。边界框通过四个坐标点确定香蕉在图像中的位置范围，类别标签则明确指示该香蕉所属的六种成熟度类别之一。精准的标注为模型提供了监督学习的依据，使其能够在训练过程中将图像特征与对应的类别准确关联起来，是模型学习的关键环节。

3.4 数据集特点与适用性

数据集具有以下显著特点：

(1) **多样性**：涵盖不同光照条件、背景环境和拍摄角度的香蕉图像。光照变化包括自然光、室内光等多种常见光源环境下的图像呈现；拍摄角度则有正面、侧面、俯视等全方位覆盖。这种多样性确保了模型经过训练后，能够在复杂多变的实际应用场景中稳定地对香蕉成熟度进行识别，无论是在香蕉的种植监测、采摘筛选还是后续的仓储、物流、销售等环节，都具有广

泛的适用性。

(2) **平衡性**：如前文所述，各类别的样本数量经过合理分配，相对均衡。在模型训练中，这种平衡性有助于避免某些类别因样本量过少而被模型忽视，或因样本量过多而使模型对该类别过度敏感，从而确保模型对六种成熟度类别都能给予同等重视，提高整体分类的准确性和稳定性，使其在面对各类别样本时都能做出公正、准确的判断。

(3) **高质量标注**：所有图像的标注均经过专业人员的严谨操作与审核，确保标注信息的准确性与完整性。精确的边界框定位和类别标记，为模型提供了清晰、可靠的学习信号，使模型能够高效地从数据中提取特征，建立正确的分类模型。高质量的标注是模型性能提升的坚实基础，直接关系到模型的最终效果。

3.5 数据集配置文件

data.yaml

```
train: D:\banana\YOLO 香蕉成熟度检测数据集\YOLO 香蕉成熟度检测数据集\images\train
val: D:\banana\YOLO 香蕉成熟度检测数据集\YOLO 香蕉成熟度检测数据集\images\val
test: D:\banana\YOLO 香蕉成熟度检测数据集\YOLO 香蕉成熟度检测数据集\images\test

# Classes
nc: 6
names: ['freshripe', 'freshunripe', 'overripe', 'ripe', 'rotten', 'unripe']
```

综上所述，本数据集规模适中且内容丰富，经过科学合理的预处理，具备良好的多样性、平衡性和标注质量，能够充分满足 YOLOv10 模型在香蕉新鲜度等级识别任务中的训练和评估需求，为模型的出色性能提供了有力的数据支撑，是实现精准、高效香蕉成熟度检测的关键因素之一。

4 环境配置

4.1 虚拟环境

创建并激活虚拟环境------yolov10

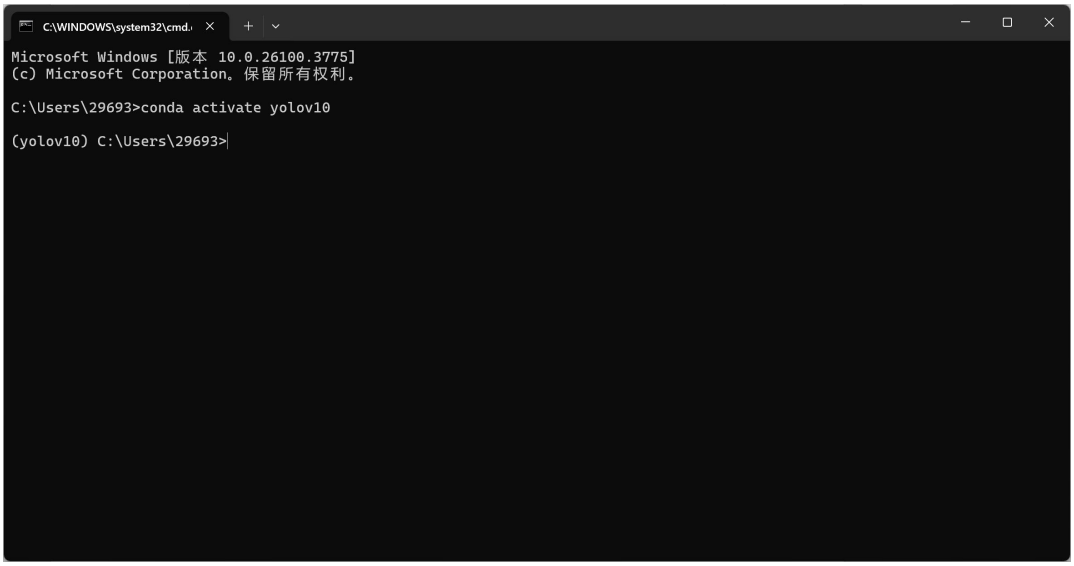


图 4-1 激活环境

4.2 安装所需库

pip install torch torchvision torchaudio

pip install -r requirements.txt

```
torch==2.0.1
torchvision==0.15.2
onnx==1.14.0
onnxruntime==1.15.1
pycocotools==2.0.7
PyYAML==6.0.1
scipy==1.13.0
onnxsim==0.4.36
onnxruntime-gpu==1.18.0
gradio==4.31.5
opencv-python==4.9.0.80
psutil==5.9.8
py-cpuinfo==9.0.0
```

```
huggingface-hub==0.23.2
safetensors==0.4.3
```

5 系统架构设计

本系统采用 C/S（客户端）架构，基于 Python 语言开发，核心功能模块包括用户登录模块、主业务功能模块（图像/视频/摄像头检测）、模型管理模块（训练与加载）及结果输出模块。系统架构层次如下：

界面层：基于 PyQt5 实现可视化交互界面，包括登录窗口与主功能窗口，负责用户输入接收与结果可视化展示。

业务逻辑层：处理用户登录验证、检测任务调度（图像/视频/摄像头）、检测结果处理（保存、显示）等核心逻辑。

模型层：集成 YOLOv10 目标检测模型，支持模型训练、加载与推理，提供香蕉新鲜度等级检测能力。

6 详细设计

6.1 模型训练

6.1.1 模型结构和调整

骨干网络：采用 YOLOv10 的 CSPDarknet 结构，通过跨阶段局部连接（CSP）减少计算量，提升特征提取效率。

颈部网络：PAFPN（路径聚合特征金字塔网络）实现多尺度特征融合，增强对不同大小香蕉的检测能力。

检测头：改进的解耦头设计，分离分类与回归分支，提升检测速度与精度。

参数调整：根据数据集特点调整锚框（Anchor Boxes），优化非极大值抑制（NMS）阈值，平衡召回率与精度。

6.1.2 训练代码

```
from ultralytics import YOLOv10

model_path = 'yolov10m.pt'
data_path = 'data.yaml'

if __name__ == '__main__':
    model = YOLOv10(model_path)
    results = model.train(data=data_path,
                          epochs=500,
                          batch=64,
                          device='0',
                          workers=0,
                          project='runs',
                          name='exp',
                          )
```

模型加载部分：使用 model = YOLOv10(model_path) 代码行加载预先准备好的 YOLOv10 模型，其中 model_path 指定了模型文件的存储路径（'yolov10m.pt'），该模型是实现目标检测功能的核心算法组件。

训练参数设置部分 ：通过 model.train() 方法设置了一系列训练相关的参数，包括：

- data=data_path：指定训练数据的配置文件路径（'data.yaml'），该数据文件包含了训练图像的路径、类别信息以及数据集的划分等关键数据，确保模型能够获取正确的训练数据进行学习。
- epochs=500：训练设置的总轮数为 500，即模型将在整个训练数据集上进行 500 次迭代训练，以充分学习数据中的特征和模式。
- batch=64：定义每次训练时批量处理的图像数量为 64 张，这在一定程度上平衡了训练速度和模型性能的稳定性。
- device='0'：指定使用编号为 0 的 GPU 设备进行训练，利用 GPU 的并行计算能力加速模型训练过程。
- workers=0：设置数据加载的线程数为 0，在本系统中根据实际硬件资源和数据集特点，选择了该配置以确保数据加载和训练过程的稳定配合。

6.1.3 训练结果

(1) 评估指标（基于测试集）

mAP@0.5：0.937（所有类别在 IOU=0.5 时的平均精度），表明模型对各新鲜度等级的检测精度较高。

类别精度：过熟（overripe）类别精度最高（0.979），腐烂（rotten）类别精度 0.907，未熟（unripe）类别精度 0.909。

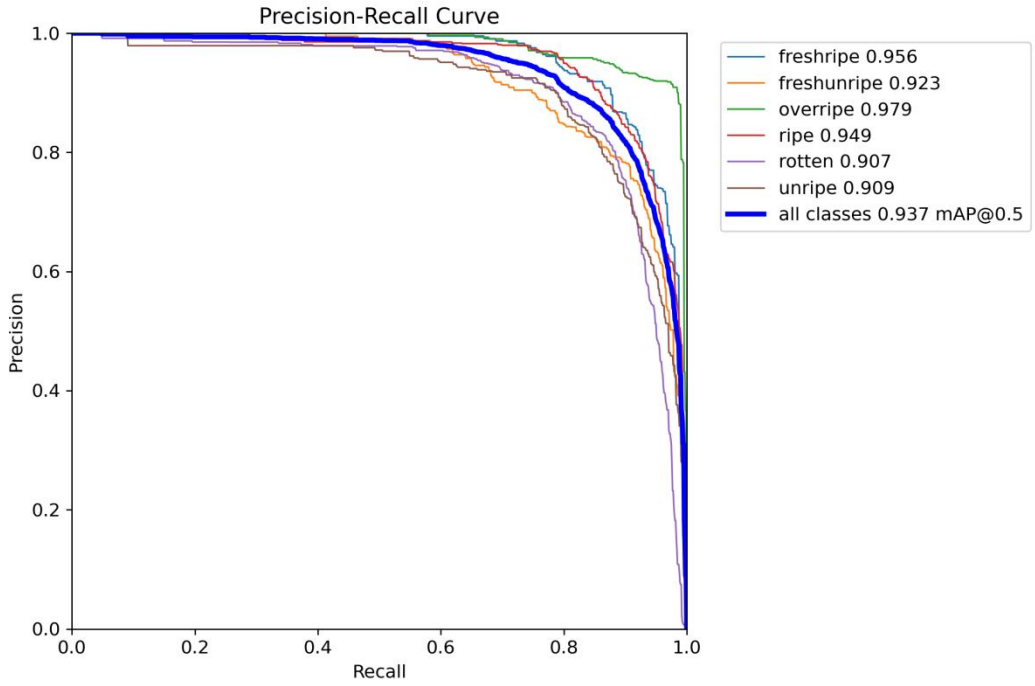


图 6-1 PR 曲线

混淆矩阵：对角线元素占比均超过 90%（如新鲜成熟类正确检测率 90%），非对角线错误主要集中在“新鲜未熟”与“未熟”等相似类别。

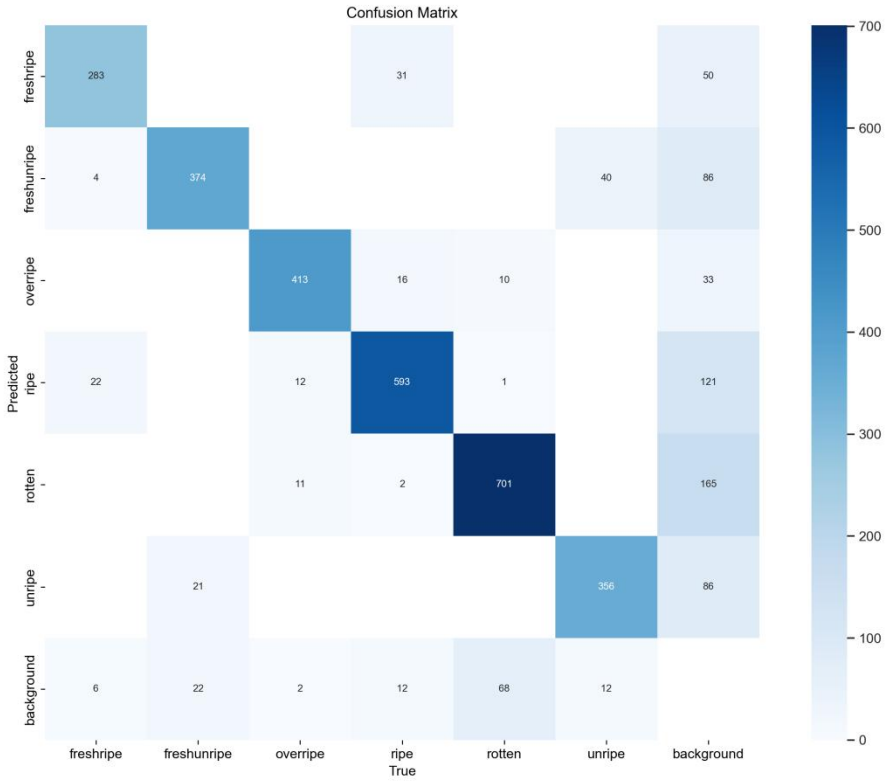


图 6-2 混淆矩阵

(2) 训练曲线

损失曲线：训练损失（box_loss、cls_loss、dfl_loss）随 epoch 增加逐步下降，验证损失在第 200epoch 后趋于稳定。

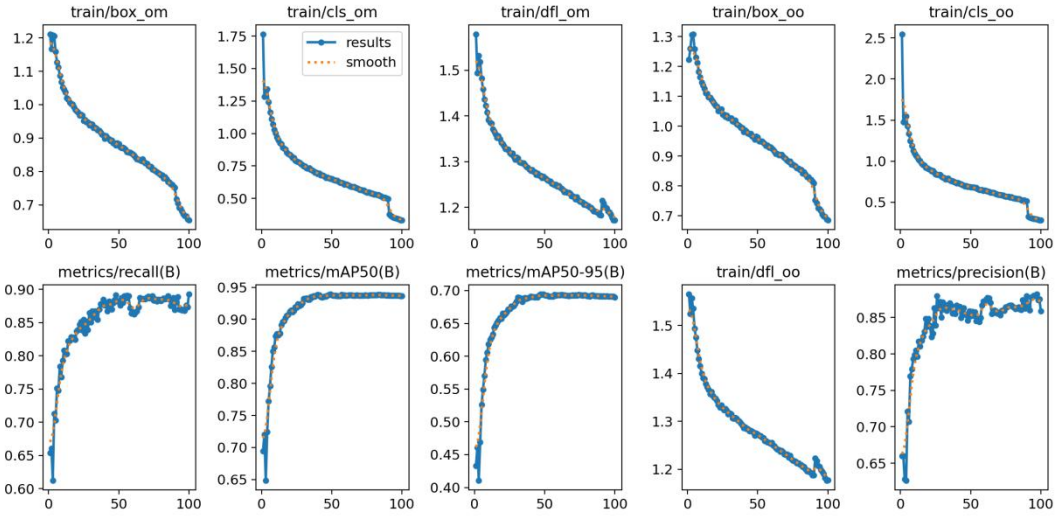


图 6-3 损失曲线

性能曲线：precision、recall 在置信度 0.5 时均超过 0.85，mAP@0.5-0.95 达到 0.693。

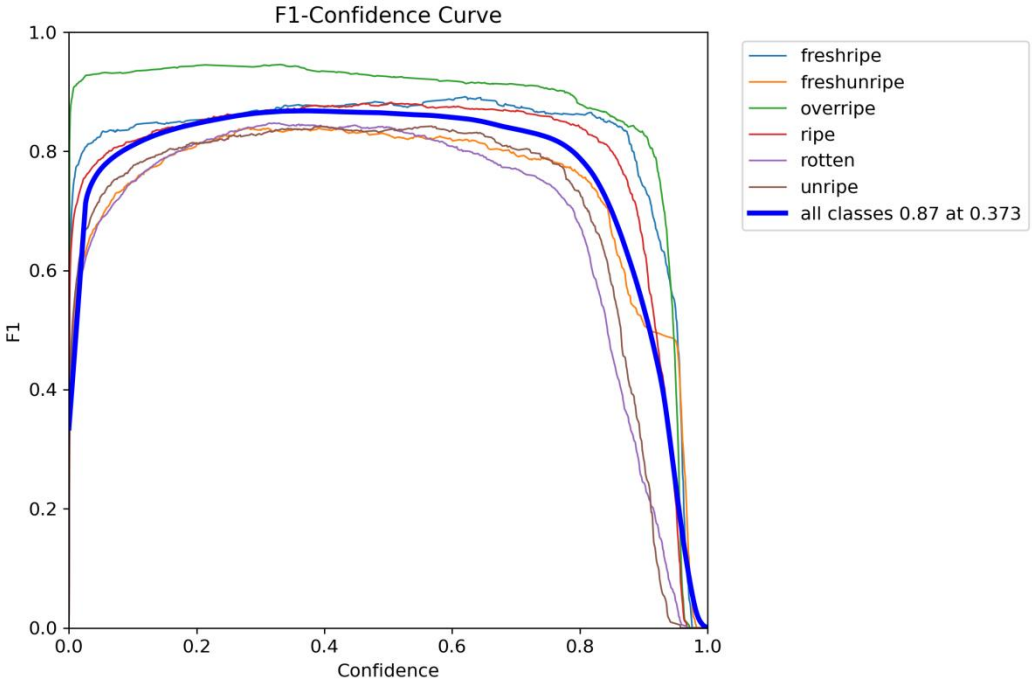


图 6-4 F1_curve

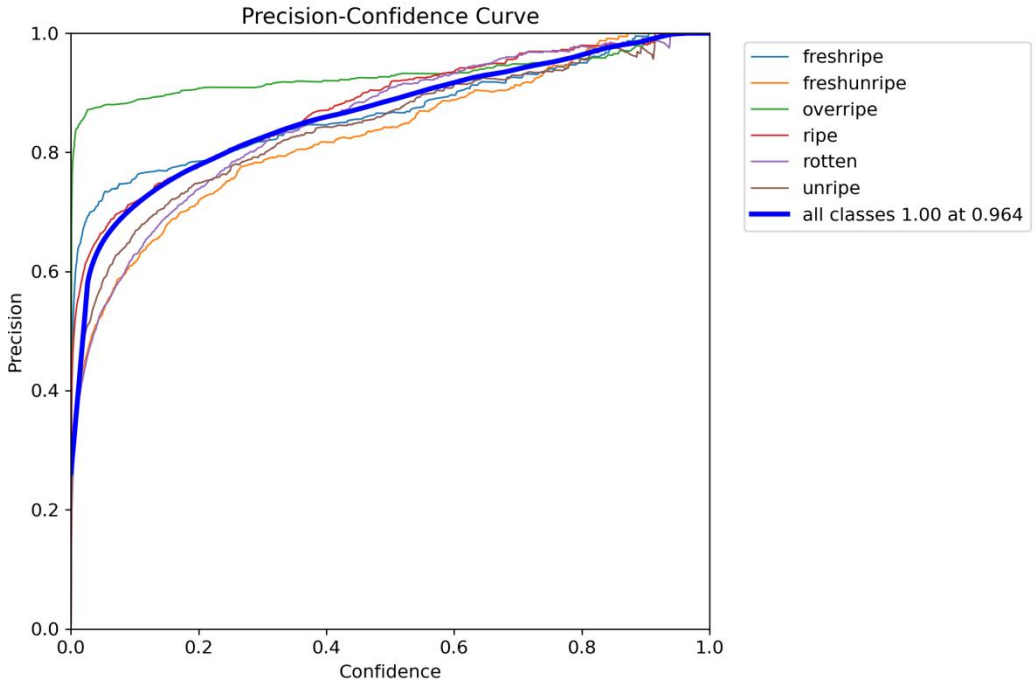


图 6-5 P_curve

6.2 核心部分

6.2.1 登录模块（LoginWindow）

功能描述：验证用户身份，仅允许合法用户进入主系统。

(1) 组成结构

类定义：LoginWindow（继承自 QWidget）。

核心方法：

init_ui()：初始化登录窗口界面布局（标题、用户名/密码输入框、登录按钮）。

set_window_style()：设置窗口整体样式（背景色、输入框/按钮样式）。

set_connections()：绑定登录按钮点击事件到验证逻辑。

check_login()：验证用户名（默认 ll）与密码（默认 2213040106），验证通过则跳转主窗口。

accept()：关闭登录窗口，启动主窗口（MainWindow）。

(2) 界面布局

窗口尺寸：固定 800×600 像素，不可缩放。

组件布局：

标题标签：居中显示“香蕉新鲜度等级识别系统”，字体为微软雅黑 20 号粗体。

输入框组：用户名与密码输入框（密码框隐藏输入内容），样式统一（圆角边框、聚焦高亮）。

登录按钮：固定高度 50px，蓝底白字，支持悬停/按下状态样式变化。

(3) 源代码（LoginWindow.py）

```
1.  import sys
2.  from PyQt5.QtWidgets import (QApplication, QWidget,
 QVBoxLayout, QHBoxLayout,
3.                                  QLabel, QLineEdit, QPu
shButton, QMessageBox)
4.  from PyQt5.QtCore import Qt
5.  from PyQt5.QtGui import QIcon, QFont
6.
7.
8.  class LoginWindow(QWidget):
9.      def __init__(self):
10.         super().__init__()
11.         self.init_ui()
12.         self.set_window_style()
13.         self.set_connections()
14.
15.     def init_ui(self):
16.         # 窗口基本设置
17.         self.setWindowTitle("系统登录")
18.          self.setWindowIcon(QIcon("D:/banana/基于深
度学习的香蕉新鲜度等级识别系统/系统图标.jpg"))
19.         self.resize(800, 600)
20.          self.setFixedSize(800, 600)  # 固定窗口大
小
21.
22.         # 主布局
23.         main_layout = QVBoxLayout(self)
24.         main_layout.setContentsMargins(50, 40, 50,
40)
25.         main_layout.setSpacing(30)
```

```
26.
27.         # 标题）
28.          title_label = QLabel("香蕉新鲜度等级识别系统
")
29.              title_label.setFont(QFont(" 微 软 雅 黑
", 20, QFont.Bold))
30.         title_label.setAlignment(Qt.AlignCenter)
31.         title_label.setStyleSheet("color: #2c3e50;")

32.         main_layout.addWidget(title_label)
33.
34.         # 用户名输入
35.         user_layout = QHBoxLayout()
36.         user_label = QLabel("用户名:")
37.         user_label.setFixedWidth(130)
38.         user_label.setFont(QFont("微软雅黑", 15))
39.         self.user_input = QLineEdit()
40.          self.user_input.setPlaceholderText("请输入
用户名")
41.         self.user_input.setStyleSheet(self.get_inpu
t_style())
42.         # 增加输入框尺寸
43.         self.user_input.setMinimumHeight(42)
44.         self.user_input.setMinimumWidth(280)
45.         user_layout.addWidget(user_label)
46.         user_layout.addWidget(self.user_input)
47.         main_layout.addLayout(user_layout)
48.
49.         # 密码输入
50.         pwd_layout = QHBoxLayout()
51.         pwd_label = QLabel("密    码:")
52.         pwd_label.setFixedWidth(130)
53.         pwd_label.setFont(QFont("微软雅黑", 15))
54.         self.pwd_input = QLineEdit()
55.          self.pwd_input.setPlaceholderText("请输入密
码")
56.         self.pwd_input.setEchoMode(QLineEdit.Passwo
rd)
57.         self.pwd_input.setStyleSheet(self.get_input
_style())
58.         # 同步输入框尺寸
59.         self.pwd_input.setMinimumHeight(42)
```

```
60.             self.pwd_input.setMinimumWidth(280)
61.             pwd_layout.addWidget(pwd_label)
62.             pwd_layout.addWidget(self.pwd_input)
63.             main_layout.addLayout(pwd_layout)
64.
65.             # 登录按钮
66.             self.login_btn = QPushButton("登录")
67.             self.login_btn.setFixedHeight(50)
68.             self.login_btn.setStyleSheet(self.get_butto
n_style())
69.             main_layout.addWidget(self.login_btn)
70.
71.     def set_window_style(self):
72.         # 窗口整体样式（输入框高度已通过单独样式控制）
73.         self.setStyleSheet("""
74.             QWidget {
75.                 background-color: #f8f9fa;
76.             }
77.         """)
78.
79.     def set_connections(self):
80.         self.login_btn.clicked.connect(self.check_l
ogin)
81.
82.     def get_input_style(self):
83.         return """
84.             QLineEdit {
85.                 border: 1px solid #e0e0e0;
86.                 border-radius: 8px;
87.                 padding: 0 15px;
88.                 font-size: 15px;
89.                 color: #34495e;
90.             }
91.             QLineEdit:focus {
92.                 border-color: #3498db;
93.                 outline: none;
94.             }
95.         """
96.
97.     def get_button_style(self):
98.         return """
99.             QPushButton {
```

```
                background-color: #3498db;
                border: none;
                color: white;
                font-size: 17px;
                font-weight: 500;
                border-radius: 8px;
            }
            QPushButton:hover {
                background-color: #2980b9;
                border: 1px solid #2573a7;
            }
            QPushButton:pressed {
                background-color: #20638f;
            }
        """

    def check_login(self):
        # 验证逻辑保持不变
        username = self.user_input.text()
        password = self.pwd_input.text()

        if username == "ll" and password == "2213040106":
            self.accept()
        else:
            QMessageBox.warning(self, "登录失败", "用户名或密码错误", QMessageBox.Ok)

    def accept(self):
        self.close()
        # 延迟导入主窗口避免循环依赖
        from main import MainWindow
        self.main_window = MainWindow()
        self.main_window.show()


if __name__ == "__main__":
    app = QApplication(sys.argv)
    login_window = LoginWindow()
    login_window.show()
    sys.exit(app.exec_())
```

6.2.2 主功能模块(MainWindow)

功能描述：提供香蕉新鲜度检测的核心操作入口（图片/视频/摄像头检测）、参数配置（置信度/IoU 阈值）及结果展示。

(1) 组成结构

类定义：MainWindow（继承自 UIWindow，由 UI.py 定义界面布局）。

核心组件：

左侧面板：显示原始图像与检测结果图像（最小尺寸 800×450 像素）。

右侧面板：包含模型设置、参数调整、功能按钮及检测结果表格。

(2) 关键子模块设计

① 模型管理模块

功能：支持 YOLOv10 模型的加载与切换（当前默认加载 best.pt）。

实现：load_model()：通过 YOLOv10 类加载指定模型（路径由 Config.py 的 model_path 配置），加载失败时弹出错误提示。

② 检测参数配置模块

功能：调整目标检测的置信度阈值（默认 0.25）与 IoU 阈值（默认 0.45）。

组件：

- 滑动条（QSlider）与数值输入框（QDoubleSpinBox）联动，滑动条范围 0-100（对应 0.0-1.0），输入框精度 0.05。
- update_confidence()/update_iou()：同步滑动条与输入框数值，并更新标签显示（如“置信度阈值:0.25”）。

③ 检测任务执行模块

功能：支持图片、视频、摄像头三种检测模式，通过多线程实现检测过程与界面解耦（避免卡顿）。

核心类：DetectionThread（继承自 QThread）。

- 输入参数：模型对象、检测源（图片路径/视频路径/摄像头 ID）、置信度阈值、IoU 阈值。
- 核心逻辑：

视频/摄像头模式：循环读取帧→模型推理→提取检测结果→通过 frame_received 信号发送原始帧、检测帧及检测结果（类别、置信度、位置）。

图片模式：单次读取图片→模型推理→发送结果后结束线程。

控制方法：stop()终止检测循环，释放资源（如摄像头、视频写入器）。

④ 检测结果展示模块

功能：实时显示检测图像，表格化展示检测结果（类别、置信度、位置）。

实现：

- display_image()：将 OpenCV 图像（BGR 格式）转换为 QPixmap，按比例缩放后显示在标签中。
- add_detection_result()：向表格插入新行，显示检测结果（居中对齐）。

⑤ 结果保存模块

功能：支持检测结果（图片/视频帧）保存到本地（默认路径 D:/banana/基于深度学习的香蕉新鲜度等级识别系统/results，由 Config.py 的 save_path 配置）。

触发方式：点击“保存结果”按钮，根据检测模式保存：

- 图片检测：保存检测结果图（result_时间戳.jpg）。
- 视频/摄像头检测：保存当前帧截图（snapshot_时间戳.jpg）。

(3) 核心代码

检测线程核心类（处理不同输入源的检测逻辑）

```
class DetectionThread(QThread):
    frame_received = pyqtSignal(np.ndarray, np.ndarray, list)  # 原始帧, 检测帧, 检测结果
    finished_signal = pyqtSignal()

    def __init__(self, model, source, conf, iou, parent=None):
```

```
            super().__init__(parent)
            self.model = model  # YOLOv10 模型
            self.source = source  # 输入源（摄像头 ID/视频路径/图片路径）
            self.conf = conf  # 置信度阈值
            self.iou = iou  # IOU 阈值
            self.running = True  # 线程运行状态控制

        def run(self):
            try:
                if isinstance(self.source, int) or self.source.endswith(('.mp4', '.avi', '.mov')):
                    # 处理视频/摄像头输入
                    cap = cv2.VideoCapture(self.source)

                    while self.running and cap.isOpened():
                        ret, frame = cap.read()
                        if not ret: break
                        self._process_frame(frame)  # 统一处理单帧检测
                        time.sleep(0.03)  # 控制 30fps
                    cap.release()
                else:
                    # 处理图片输入
                    frame = cv2.imread(self.source)
                    if frame is not None:
                        self._process_frame(frame)  # 统一处理单帧检测
            except Exception as e:
                print(f"Detection error: {e}")
            finally:
                self.finished_signal.emit()

        def _process_frame(self, frame):
            """统一处理单帧检测逻辑"""
            original_frame = frame.copy()
            results = self.model(frame, conf=self.conf, iou=self.iou)  # 模型推理
            annotated_frame = results[0].plot()  # 绘制检测框
```

```
            # 提取检测结果（类别、置信度、坐标）
            detections = []
            for box in results[0].boxes:
                class_name = self.model.names[int(box.cls)]
                confidence = float(box.conf)
                x, y, w, h = box.xywh[0].tolist()  # 中心坐标+宽高
                detections.append((class_name, confidence, x, y))

            # 发送信号（转换为 RGB 格式用于 UI 显示）
            self.frame_received.emit(
                cv2.cvtColor(original_frame, cv2.COLOR_BGR2RGB),
                cv2.cvtColor(annotated_frame, cv2.COLOR_BGR2RGB),
                detections
            )

    def stop(self):
        self.running = False  # 终止线程运行
```

主窗口核心逻辑（用户交互与检测流程控制）

```
class MainWindow(UIWindow):
    def __init__(self):
        super().__init__()
        self.model = None  # YOLOv10 模型实例
        self.detection_thread = None  # 检测线程实例
        self.video_writer = None  # 视频保存写入器
        self.last_detection_result = None  # 最后一次检测结果

        # 初始化模型（根据 UI 选择加载）
        self.load_model()

    def load_model(self):
        """加载 YOLOv10 模型"""
        model_name = self.model_combo.currentText()  # 从 UI 下拉框获取模型名
```

```
            self.model = YOLOv10(f"{model_name}.pt")  # 加载本地或自动下载模型

        def detect_image(self):
            """图片检测入口"""
            file_path, _ = QFileDialog.getOpenFileName(self, "选择图片", "", "图片文件 (*.jpg *.jpeg *.png *.bmp)")
            if file_path:
                self._start_detection_thread(file_path)  # 启动检测线程

        def detect_video(self):
            """视频检测入口"""
            file_path, _ = QFileDialog.getOpenFileName(self, "选择视频", "", "视频文件 (*.mp4 *.avi *.mov)")
            if file_path:
                # 初始化视频保存写入器（根据原视频参数）
                cap = cv2.VideoCapture(file_path)
                fps = cap.get(cv2.CAP_PROP_FPS)
                frame_size = (int(cap.get(cv2.CAP_PROP_FRAME_WIDTH)), int(cap.get(cv2.CAP_PROP_FRAME_HEIGHT)))
                cap.release()
                self.video_writer = cv2.VideoWriter(
                    f"results/result_{time.strftime('%Y%m%d_%H%M%S')}.mp4",
                    cv2.VideoWriter_fourcc(*'mp4v'), fps, frame_size
                )
                self._start_detection_thread(file_path)  # 启动检测线程

        def detect_camera(self):
            """摄像头检测入口"""
              self._start_detection_thread(0)  # 摄像头ID=0

        def _start_detection_thread(self, source):
            """统一启动检测线程的逻辑"""
            if self.detection_thread and self.detection_thread.isRunning():
```

```
                QMessageBox.warning(self, "警告", "请先停止当前检测任务")
                return
            conf = self.confidence_spinbox.value()  # 从 UI 获取置信度阈值
            iou = self.iou_spinbox.value()  # 从 UI 获取 IOU 阈值
            self.detection_thread = DetectionThread(self.model, source, conf, iou)
            self.detection_thread.frame_received.connect(self.on_frame_received)
            self.detection_thread.finished_signal.connect(self.on_detection_finished)
            self.detection_thread.start()

        def stop_detection(self):
            """停止检测并清理资源"""
            if self.detection_thread and self.detection_thread.isRunning():
                self.detection_thread.stop()  # 终止线程运行
                self.detection_thread.quit()
                self.detection_thread.wait()
            if self.video_writer:
                self.video_writer.release()  # 释放视频写入器
                self.video_writer = None

        def on_frame_received(self, original_frame, result_frame, detections):
            """检测结果回调（更新 UI 和保存数据）"""
            self.display_image(self.original_image_label, original_frame)  # 显示原始帧
            self.display_image(self.result_image_label, result_frame)  # 显示检测帧
            self.last_detection_result = result_frame  # 保存最新结果
            self._update_detection_table(detections)  # 更新检测结果表格
            if self.video_writer:
                self.video_writer.write(cv2.cvtColor(result_frame, cv2.COLOR_RGB2BGR))  # 写入视频帧
```

```
72.
73.     def save_result(self):
74.         """保存检测结果（图片/截图）"""
75.         if self.last_detection_result is None:
76.             QMessageBox.warning(self, "警告", "没有可保存的检测结果")
77.             return
78.         save_path = f"results/snapshot_{time.strftime('%Y%m%d_%H%M%S')}.jpg"
79.         cv2.imwrite(save_path, cv2.cvtColor(self.last_detection_result, cv2.COLOR_RGB2BGR))
```

7 结果展示

7.1 登陆窗口



图 7-1 登陆界面

交互逻辑：

- 输入用户名/密码后点击“登录”，验证失败弹出警告框（“用户名或密码错误”）。
- 验证成功关闭当前窗口，启动主窗口。



图 7-2 失败警告窗口

7.2 主功能窗口

界面布局：

左侧：上下排列两个分组框（“原始图像”“检测结果”），分别显示输入图像与模型标注后的结果图像。

右侧：

- 模型设置：下拉框选择已训练模型（默认 best）。
- 参数调整：滑动条与输入框联动设置置信度/IoU 阈值。
- 功能按钮：图片检测、视频检测、摄像头检测、停止检测、保存结果（样式统一为蓝色圆角按钮）。
- 检测结果表格：显示检测到的香蕉类别、置信度及位置（x,y 坐标）。

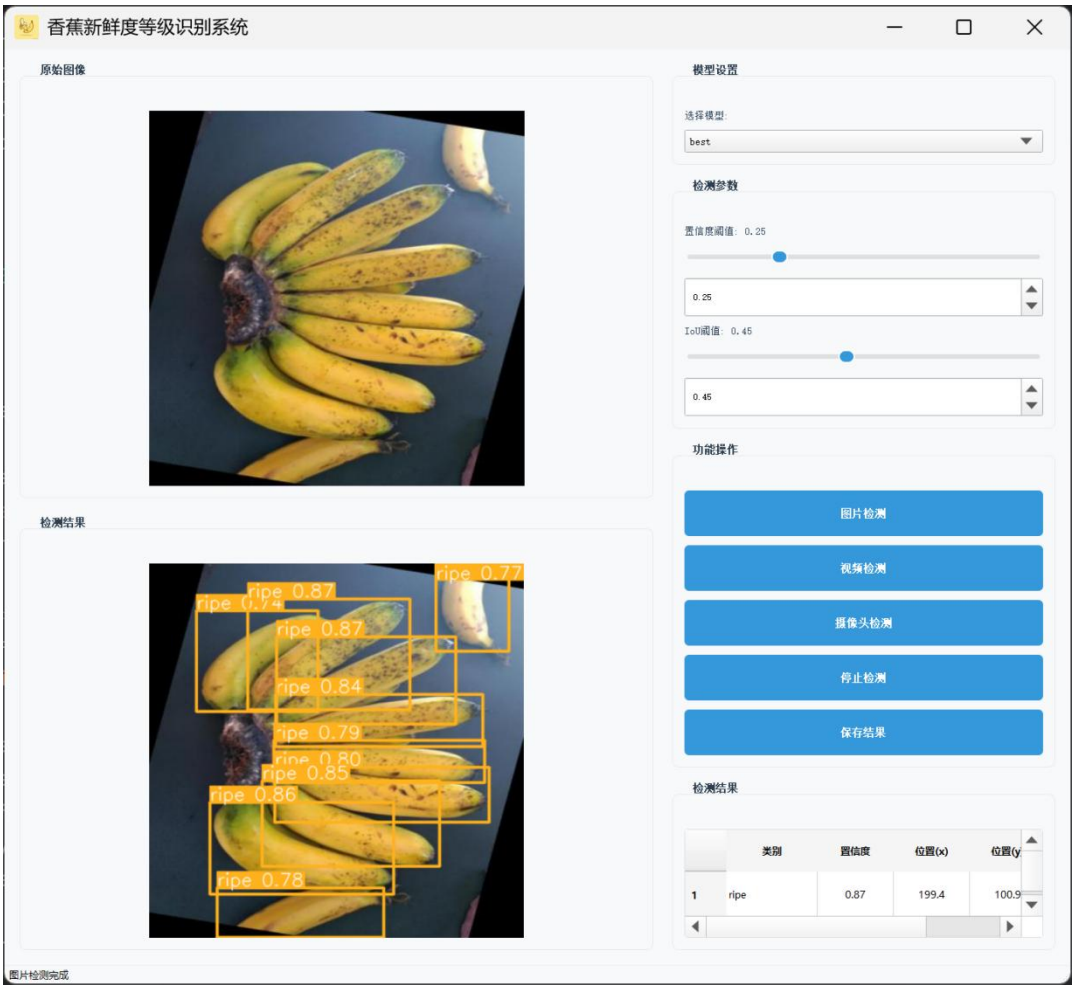

香蕉新鲜度等级识别系统
原始图像
检测结果
ripe 0.87
ripe 0.87
ripe 0.84
ripe 0.79
ripe 0.85
ripe 0.86
ripe 0.78
ripe 0.77
模型设置
选择模型:
best
检测参数
置信度阈值: 0.25
0.25
IoU阈值: 0.45
0.45
功能操作
图片检测
视频检测
摄像头检测
停止检测
保存结果
检测结果
类别
置信度
位置(x)
位置(y
1
ripe
0.87
199.4
100.9
图片检测完成

图 7-3 图片检测

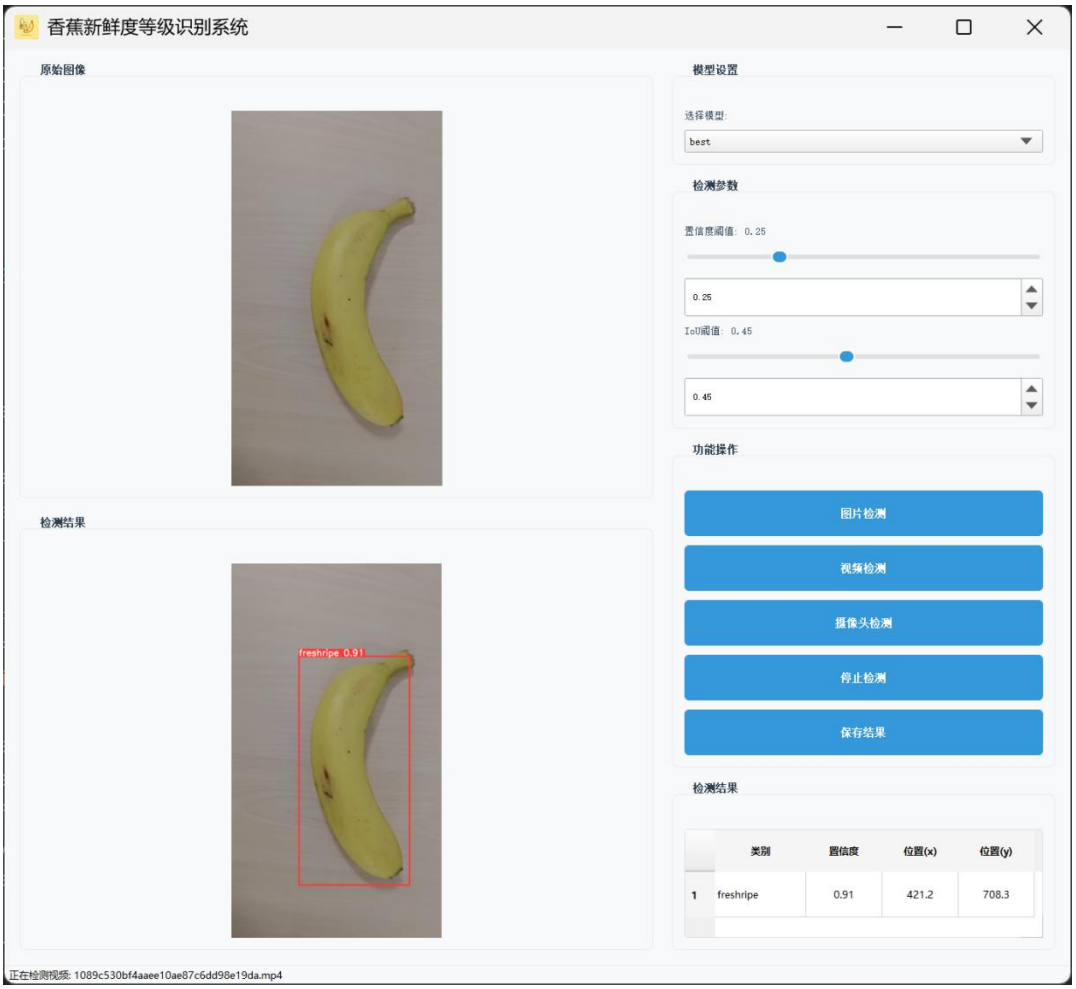

香蕉新鲜度等级识别系统
原始图像
检测结果
freshripe 0.91
模型设置
选择模型:
best
检测参数
置信度阈值: 0.25
0.25
IoU阈值: 0.45
0.45
功能操作
图片检测
视频检测
摄像头检测
停止检测
保存结果
检测结果
类别
置信度
位置(x)
位置(y)
1
freshripe
0.91
421.2
708.3
正在检测视频: 1089c530bf4aaee10ae87c6dd98e19da.mp4

图 7-4 视频检测

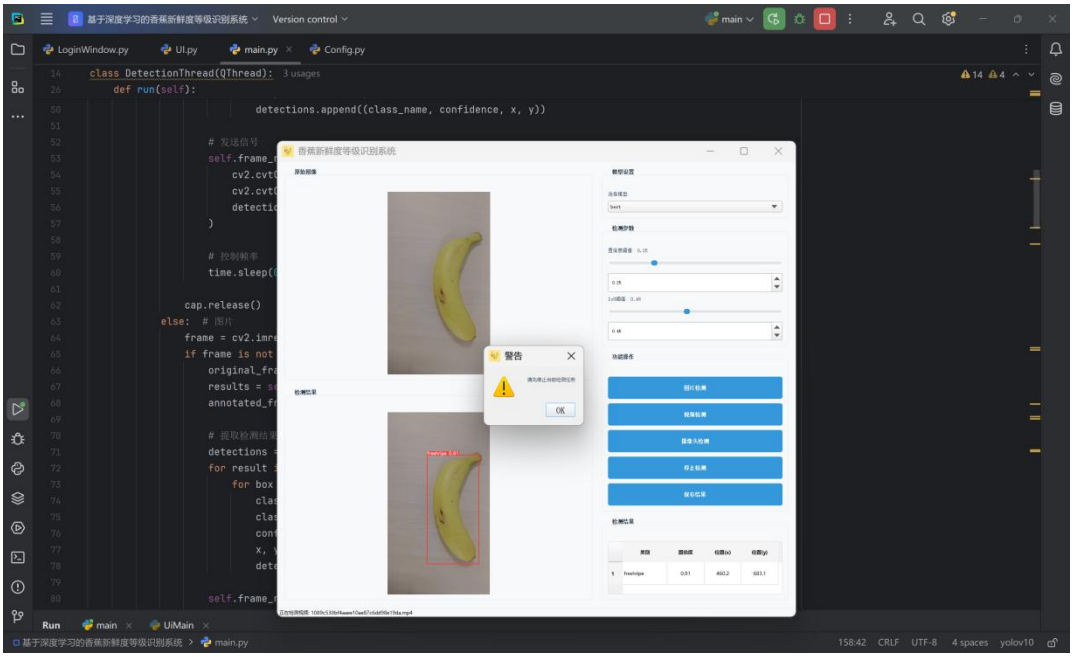


图 7-5 任务未结束警告框

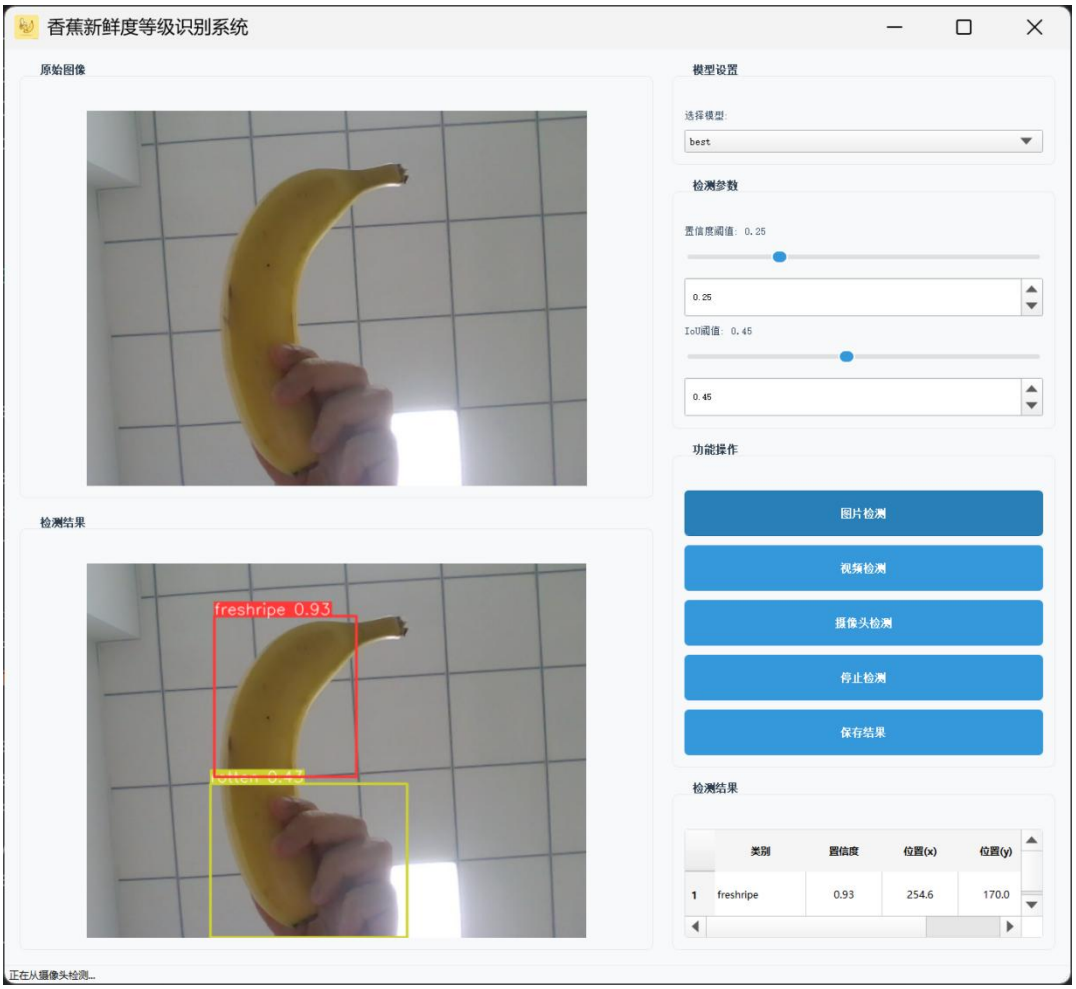


图 7-6 视频检测

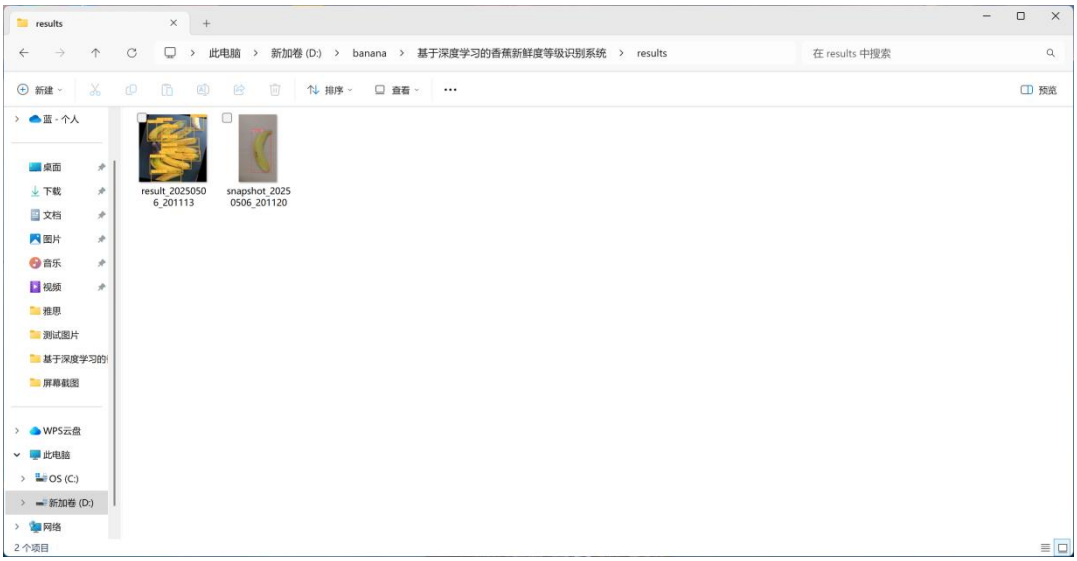


图 7-7 保存结果

8 项目成果总结

本系统通过 YOLOv10 目标检测技术实现了香蕉新鲜度的自动化等级识别。

(1) 具备以下优势：

高精度：在测试集上 mAP@0.5 达 0.937，能够准确区分 6 种新鲜度等级，适用于复杂场景。

实时性：支持摄像头实时检测，帧率达 30FPS，满足流水线快速检测需求。

易用性：图形化界面支持多模态输入、参数调节与结果保存，操作便捷。

(2) 应用场景

香蕉采收后的初步分级筛选。

仓储环境中的新鲜度监测。

电商平台的商品品质审核。

(3) 改进方向

引入多光谱图像数据，结合颜色、纹理以外的光谱特征提升检测鲁棒性。

优化模型轻量化，支持移动端部署（如 Android/iOS 设备）。

集成物联网设备，实现香蕉存储环境（温湿度、气体浓度）与新鲜度的联合监测。

通过本系统，可显著提升香蕉新鲜度检测的效率与准确性，为农业与食品行业的智能化提供有效解决方案。